

Databricks 14 Days AI Challenge

Day 5: Delta Lake Advanced

1. Time Travel

This is one of Delta's strongest features.

Every time you: append, overwrite, merge and delete

Delta creates a **new version** in `_delta_log`.

Check version history

```
DESCRIBE HISTORY ecommerce.sales;
```

You'll see version number, timestamp, operation (WRITE, MERGE, DELETE) and user

Example output:

version	operation	timestamp
0	CREATE TABLE	...
1	WRITE	...
2	MERGE	...

Read old version

By version number

```
spark.read.format("delta") \
  .option("versionAsOf", 1) \
  .table("ecommerce.sales")
```

By timestamp

```
spark.read.format("delta") \
  .option("timestampAsOf", "2026-01-20") \
  .table("ecommerce.sales")
```

Why these matters

Real-world use: ETL bug detected, Wrong data loaded and Analyst says numbers changed

Instead of panic: Query older version, Compare and Restore if needed

Restore table

RESTORE TABLE ecommerce.sales TO VERSION AS OF 1;

That's rollback in one command.

2. Merge operation (Upserts)

This is extremely important in data engineering.

Problem

You receive new daily data:

- Some records are new
- Some records already exist and must be updated

Parquet can't do this cleanly.

Delta can.

Example scenario

id	name	city
----	------	------

1	Rahul	Delhi
---	-------	-------

2	Anita	Mumbai
---	-------	--------

Incoming data:

id	name	city
----	------	------

2	Anita	Pune
---	-------	------

3	John	Bangalore
---	------	-----------

MERGE logic

- If id matches → UPDATE
- If id not found → INSERT

Delta MERGE

```
sql
```

```
MERGE INTO ecommerce.customers AS target
USING updates AS source
ON target.id = source.id
```

```
WHEN MATCHED THEN
  UPDATE SET *
```

```
WHEN NOT MATCHED THEN
  INSERT *
```

id	name	city
----	------	------

1	Rahul	Delhi
---	-------	-------

2	Anita	Pune
---	-------	------

3	John	Bangalore
---	------	-----------

Why MERGE is powerful

Used for:

- Slowly Changing Dimensions
- CDC pipelines
- Incremental ingestion
- Real-time updates

Interview gold topic.

3. Optimize & Zorder

As data grows, files become messy.

Delta writes: many small files and scattered data

i.e., small files = slow queries.

OPTIMIZE

Compacts small files into larger ones.

```
OPTIMIZE ecommerce.sales;
```

What it does:

- merges small Parquet files
- improves scan efficiency

ZORDER

Reorganizes data by a column.

```
OPTIMIZE ecommerce.sales
```

```
ZORDER BY (customer_id);
```

What this does:

- clusters similar customer_id values together
- reduces data scanned during queries

Very useful when:

- filtering on specific columns often
 - large tables (billions of rows)
-

When to use ZORDER

If queries often run like:

```
SELECT * FROM sales WHERE customer_id = 123;
```

ZORDER helps massively.

4. VACUUM (Cleanup)

Delta keeps old versions for time travel.

That means:

- old files still exist
- storage grows

VACUUM removes old files.

Default retention

7 days.

Run VACUUM

```
VACUUM ecommerce.sales;
```

Removes files older than retention threshold.

Set custom retention

```
VACUUM ecommerce.sales RETAIN 168 HOURS;
```

(168 hours = 7 days)

Important

Once vacuumed:

- old versions cannot be restored
- time travel limited

Never vacuum aggressively in production without policy.